

Industrial AI Roadmap & The Role of AI Orchestrators

A Pragmatic View from the Trenches

Adhi Rahmadian, S.Si, M.Si



The AI Landscape: Three Paradigms

DIMENSION	Classical ML	Deep Learning	GenAI
Nature	Deterministic / Statistical	Pattern Recognition	Probabilistic Generation
Training Data	Structured, labeled datasets	Large labeled datasets	Massive unlabeled corpora
Use Cases	Classification, regression, forecasting	Computer vision, NLP tasks	Content creation, summarization
Interpretability	High (feature importance)	Low (black box)	Very Low (emergent behavior)

The AI Orchestrator: Traffic Control for Models

Definition: System-level component that manages multiple AI models and routes requests intelligently

Core Functions:

- Routing: Select optimal model based on task complexity, cost, latency requirements
- Guardrails: Input validation, output filtering, safety checks, PII detection
- Fallback Logic: Graceful degradation when primary models fail
- Monitoring: Track performance, cost, and quality metrics

Think of it as: API gateway + decision engine + safety layer

When Bigger Isn't Better

Cost Reality

GPT-4 API call = \$0.03-0.12 per 1K tokens; Classical model = fractions of a cent

Latency

Transformer inference = 200-2000ms; Decision tree = <10ms

Explainability

Regulatory requirements (finance, healthcare) demand interpretable models

Maintenance

Model drift, retraining costs, dependency on external APIs

Match the tool to the problem, not the hype cycle

The Messy Reality: Data & Compute Constraints

Dirty Data in Industry

- 60-80% of ML project time spent on data cleaning (not modeling)
- Missing values, inconsistent formats, duplicate records, label noise
- Legacy systems with undocumented schemas

Compute Limitations

- GPU scarcity: H100 waitlists measured in months
- Cloud costs: Training a large model = \$100K-\$2M
- Inference at scale: Serving millions of users requires optimization (quantization, distillation)

The gap: Academic datasets (clean, benchmarked) vs. production data (chaos)

Case Study: Hype vs. Reality

Marketing Claim

"AI-Powered Automated Coding: Developers Write 80% Less Code!"

Engineering Reality:

- What it actually does: Context-aware autocomplete and boilerplate generation
- Developer still writes: Architecture, logic, tests, and debugging
- Works well for: Standard CRUD operations, common patterns, syntax completion
- Fails at: Novel algorithms, domain-specific logic, performance optimization
- Real productivity gain: 15-30% time savings on routine tasks

AI augments expertise, doesn't replace it (yet)